

1 实验目的

- 认识和掌握多周期数据通路图的构成、原理及其设计方法
- 掌握多周期 CPU 的实现方法，代码实现方法
- 编写一个编译器，将 MIPS 汇编程序编译为二进制机器码
- 掌握多周期 CPU 的测试方法
- 掌握多周期 CPU 的实现方法

2 实验内容

使用 Vivado 开发平台设计一个单周期 CPU, 使其能够满足至少下列 16 条指令:

1. add rd rs rt

000000	rs (5 位)	rt (5 位)	rd (5 位)	00000 100000
--------	----------	----------	----------	--------------

2.sub rd rs rt

000000	rs (5 位)	rt (5 位)	rd (5 位)	00000 100010
--------	----------	----------	----------	--------------

3.addiu rt rs immediate

001001	rs (5 位)	rt (5 位)	immediate(16 位)
--------	----------	----------	-----------------

4.andi rt rs immediate

001100	rs (5 位)	rt (5 位)	immediate(16 位)
--------	----------	----------	-----------------

5.and rd rs rt

000000	rs (5 位)	rt (5 位)	rd (5 位)	00000 100100
--------	----------	----------	----------	--------------

6.ori rt rs immediate

001101	rs (5 位)	rt (5 位)	immediate(16 位)
--------	----------	----------	-----------------

7.or rd rs rt

000000	rs (5 位)	rt (5 位)	rd (5 位)	00000 100101
--------	----------	----------	----------	--------------

8.sll rd rt sa

000000	未用	rt (5 位)	rd (5 位)	sa(5 位)	000000
--------	----	----------	----------	---------	--------

9.slti rt rs immediate

001010	rs (5 位)	rt (5 位)	immediate(16 位)
--------	----------	----------	-----------------

10.sw rt offset(rs)

101011	rs(5 位)	rt(5 位)	offset(16 位)
--------	---------	---------	--------------

11.lw rt offset(rs)

100011	rs(5 位)	rt(5 位)	offset(16 位)
--------	---------	---------	--------------

12.beq rs rt offset

000100	rs (5 位)	rt (5 位)	offset(16 位)
--------	----------	----------	--------------

13.bne rs rt offset

000101	rs (5 位)	rt (5 位)	offset(16 位)
--------	----------	----------	--------------

14.blez rs offset

000110	rs (5 位)	00000	offset(16 位)
--------	----------	-------	--------------

15.j addr

000010	addr(26 位)
--------	------------

16.halt

111111	00000000000000000000000000000000(26 位)
--------	--

实验补充要求:

- 模块设计的思想、方法: 流程图, 控制信号表的作用, 如何应用? 对于实验课件中未提供完整源代码的模块, 需要提供相关实现代码说明 (代码部分必须有侧重点, 无需太泛!), 强调关键部分, 并辅以必要的文字说明。
- 截取波形图和在板上实际运行的结果进行说明。对于实验课件中提供的每个测试输入, 实验报告中需要展示包含与该输入相关的、能说明该输入被正确处理的信号 (控制信号、结果数据、寄存器内容和地址等) 的波形图以及板上运行结果截图, 并辅以必要的文字说明。

将设计好的 CPU 烧到开发板上, 满足以下要求:

- 开关 SW_in (SW15, SW14) 控制七段数码管的显示内容, 当 SW_in =
 - 00 时, 显示当前 PC 值: 下条指令的 PC 值;
 - 01 时, 显示 RS 寄存器地址:RS 寄存器数据;
 - 10 时, 显示 RT 寄存器地址:RT 寄存器数据;
 - 11 时, 显示 ALU 结果输出:DB 总线数据。
- 指令执行采用单步 (按键控制) 执行方式, 由开关 (SW15,SW14) 控制选择查看数码管上的相关信息、地址和数据。地址或数据的输出经过模块代码转换到数码管上。
- 复位信号 (reset) 接开关 SW0, 按键 (单脉冲) 接按键 BTNR。
- 指令执行采用单步执行方式, 由开关 (SW15、SW14) 控制选择查看数码管上的相关信息, 地址和数据。地址或数据的输出经以下模块代码转换后接到数码管上。

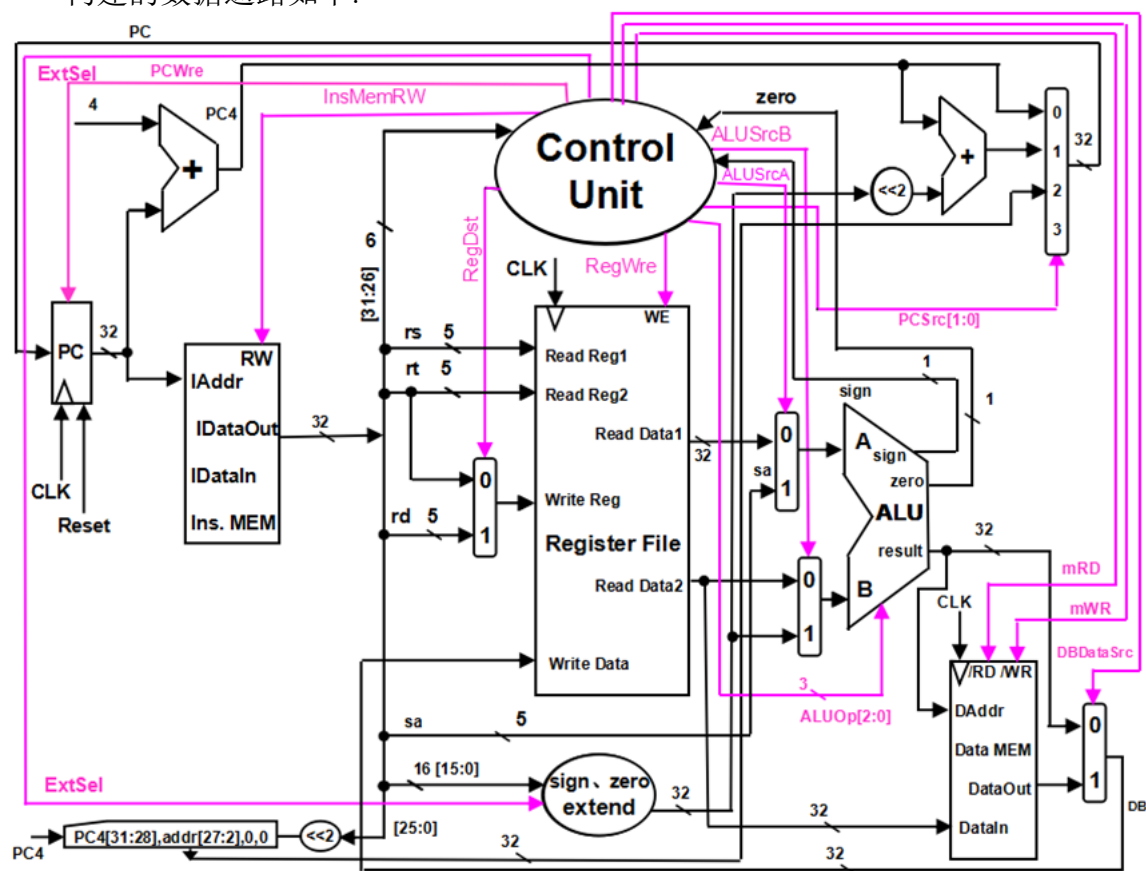
3 实验原理

3.1 单周期 CPU 的原理

单周期 CPU 的原理是通过一个时钟周期内完成一条指令的执行, 将指令的取指、译码、执行、访存和写回等操作串行组合在一个完整的时钟周期中。在每个时钟周期内, 所有的部件 (如寄存器堆、ALU、数据存储器等) 协同工作, 确保一条指令从开始到结束的所有步骤都在一个时钟脉冲内完成。这种架构中, 每条指令都是独立执行的, 执行顺序简单明确。

3.2 数据通路的构建

构建的数据通路如下:



其中 CPU 接受两个输入: 周期一定的时钟输入和清零信号 Reset, 其余各组件的主要功能如下:

- 指令存储器存储要执行的指令
- PC 决定读取指令的地址
- 寄存器堆有 32 个寄存器, 接受读写信号进行寄存器的读和写
- 数据存储器存储数据, 接受 lw 和 sw 信号进行数据的存储和加载
- ALU 算术逻辑单元进行计算
- 符号扩展单元将 16 位立即数进行零扩展或符号扩展

- 5 线和 32 线双选器作为数据通路的重要部分, 决定了 ALU 的输入, 寄存器写等功能
- 控制单元提供控制信号, 使得 CPU 根据各种不同的指令执行各自的功能

3.3 控制信号的解释

其中各个控制信号以及其对应状态的功能如下:

表 1: 控制信号对应功能

控制信号名 \ 状态	0	1
PC_Wre	时钟上升沿到来时不写 PC	时钟上升沿到来时写 PC
ALUsrcA	ALU 的 A 口数据来自 data1	ALU 的 A 口数据来自移位数 (sa)
ALUsrcB	ALU 的 B 口数据来自 data2	ALU 的 B 口数据来自符号扩展后的立即数
Dat_Src	写入寄存器的数据来自 ALU	写入寄存器的数据来自存储器
Reg_Wre	不写寄存器	写寄存器
Ins_Mem_RD	写指令寄存器	不写指令寄存器
Mem_RD	输出高阻态	读数据存储器
Mem_Wre	不写数据存储器	写数据存储器
Reg_Dst	目的寄存器是 rt	目的寄存器是 rd
ExtOP	进行 0 扩展	进行符号扩展

而 ALUOP 的信号以及功能如下:

- 000: add
- 001: sub
- 010: $B \ll A$
- 011: $A|B$
- 100: $A \& B$
- 101: $A < B ? 1 : 0$ (无符号比较)
- 110: 带符号比较
- 111: $A \oplus B$

PC_Src 的信号以及功能如下:

- 00: PC+4
- 01: $PC+4+((\text{sign-extend})\text{immediat}e \ll 2)$

- 10: {(PC+4)[31:28],instruction[27:2],2'b00}
- 11: default

其中各个指令对应的控制信号如下:

表 2: 各个指令对应的控制信号												
指令 \ 控制信号	PC_Wre	ALUsrcA	ALUsrcB	Data_Src	Reg_Wre	Ins_Mem_RD	Mem_RD	Mem_Wre	Reg_Dst	ExtOP	ALUOP	PC_Src
add	1	0	0	0	1	1	x	0	1	x	000	00
sub	1	0	0	0	1	1	x	0	1	x	001	00
addiu	1	0	1	0	1	1	x	0	0	1	000	00
andi	1	0	1	0	1	1	x	0	0	0	100	00
and	1	0	0	0	1	1	x	0	1	x	100	00
ori	1	0	1	0	1	1	x	0	0	0	011	00
or	1	0	0	0	1	1	x	0	1	x	011	00
sll	1	1	0	0	1	1	x	0	1	x	010	00
slti	1	0	1	0	1	1	x	0	0	1	110	00
sw	1	0	1	x	0	1	x	1	x	1	000	00
lw	1	0	1	1	1	1	1	0	0	1	000	00
beq	1	0	0	x	0	1	x	0	x	1	001	zero?01:00
j	1	x	x	x	0	1	x	0	x	x	x	10
halt	0	x	x	x	x	x	x	x	x	x	x	x
bne	1	0	0	x	0	1	x	0	x	1	001	zero?00:01
blez	1	0	0	x	0	1	x	0	x	1	001	sign zero?01:00

4 实验器材

- 电脑一台
- Xilinx Vivado 软件一套
- Basys3 板一块

5 实验过程与结果

在 CPU 的设计中, 由于各组件的功能比较复杂,IO 较多, 因此在 verilog 编程中采用分模块设计的方法, 先给每个组件设计底层模块, 最后再用一个顶层模块将它们综合起来, 进行协调运作。

5.1 底层模块的设计

五线双选器:

```

1  `timescale 1ns / 1ps
2  module mul_5(
3      input  [4:0]A,
4      input  [4:0]B,
5      input op,
6      output [4:0]out
7  );

```

```

8         assign out=(op==0)?A:B;
9     endmodule

```

1: mul_5.v

32 线双选器:

```

1     `timescale 1ns / 1ps
2     module mul_32(
3         input  [31:0]A,
4         input  [31:0]B,
5         input  op,
6         output [31:0]out
7     );
8         assign out=(op==0)?A:B;
9     endmodule

```

2: mul_32.v

符号扩展器:

将 16 位立即数扩展位 32 位

```

1     `timescale 1ns / 1ps
2     module Ext_16_to_32(
3         input  [15:0]immediate,
4         input  Extop,
5         output [31:0]immediate_32
6     );
7         assign immediate_32 = (Extop && immediate[15]) ? {16'b1111111111111111, immediate} :
            {16'b0000000000000000, immediate};
8     endmodule

```

3: Ext_16_to_32.v

ALU 算术逻辑单元:

根据 ALUOP 和输入的 A、B, 进行相应的运算

```

1     `timescale 1ns / 1ps
2     module ALU(
3         input  [31:0]A,
4         input  [31:0]B,
5         input  [2:0]ALUOP,
6         output reg[31:0]ans,
7         output Zero,
8         output Sign
9     );
10        always@(*) begin
11            case(ALUOP)
12                3'b000: ans=A+B;
13                3'b001: ans=A-B;
14                3'b010: ans=B<<A;
15                3'b011: ans=A|B;

```

```

16         3'b100: ans=A&B;
17         3'b101: ans=(A<B?1:0);
18         3'b110: ans=(A[31]!=B[31]?(A[31]==1?1:0):A<B);
19         3'b111: ans=A^B;
20     endcase
21 end
22 assign Zero=(ans==0);
23 assign Sign=ans[31];
24 endmodule

```

4: ALU.v

控制器:

根据上述表格, 给不同的指令赋予各自的控制信号

```

1  `timescale 1ns / 1ps
2  module Control(
3      input Zero,
4      input Sign,
5      input [5:0]OP,
6      input [5:0]func,
7      output PC_Wre,
8      output ALUsrcA,
9      output ALUsrcB,
10     output Data_Src,
11     output Reg_Wre,
12     output Ins_Mem_RD,
13     output Mem_RD,
14     output Mem_Wre,
15     output Reg_Dst,
16     output ExtOP,
17     output [2:0]ALUOP,
18     output [1:0]PC_Src
19 );
20     parameter R=6'b000000;
21     parameter add=6'b100000;
22     parameter sub=6'b100010;
23     parameter And=6'b100100;
24     parameter Or=6'b100101;
25     parameter sll=6'b000000;
26     parameter addiu=6'b001001;
27     parameter andi=6'b001100;
28     parameter ori=6'b001101;
29     parameter slti=6'b001010;
30     parameter sw=6'b101011;
31     parameter lw=6'b100011;
32     parameter j=6'b000010;
33     parameter halt=6'b111111;
34     parameter beq=6'b000100;
35     parameter bne=6'b000101;
36     parameter blez=6'b000110;
37
38     assign PC_Wre= OP!=halt;

```

```

39     assign ALUSrcA= OP==R && func==sll;
40     assign ALUSrcB= OP==addiu||OP==andi||OP==ori||OP==slti||OP==sw||OP==lw;
41     assign Data_Src= OP==lw;
42     assign Reg_Wre= (OP!=sw && OP!=beq && OP!=j && OP !=bne && OP!=blez)?1:0;
43     assign Ins_Mem_RD=1;
44     assign Mem_RD= OP==lw;
45     assign Mem_Wre= OP==sw;
46     assign Reg_Dst=OP==R;
47     assign ExtOP= OP!=andi && OP!=ori;
48     assign ALUOP= (OP==R&&func==sub) || OP==beq || OP==bne || OP==blez ?3'b001
49                  : (OP==R&&func==add) || OP==addiu || OP==sw || OP==lw ?3'b000
50                  : OP == andi||(OP==R&&func==And)?3'b100
51                  : OP==ori||(OP==R&&func==Or)?3'b011
52                  : (OP==R&&func==sll)?3'b010
53                  : OP==slti?3'b110
54                  : 3'b000;
55     assign PC_Src= OP==j?2'b10
56                  : OP==beq?Zero?2'b01:2'b00
57                  : OP==bne?Zero?2'b00:2'b01
58                  : OP==blez?Sign?2'b01:2'b00
59                  : 2'b00;
60     endmodule

```

5: Control.v

数据存储器:

根据读/写的地址和使能, 对内存进行读/写, 按字节编址, 采用大端方式存储

```

1  `timescale 1ns / 1ps
2  module Data_mem(
3      input CLK,
4      input RD_EN,
5      input W_EN,
6      input [31:0]R_addr,
7      input [31:0]W_addr,
8      input [31:0]W_data,
9      output reg [31:0]R_data
10 );
11     reg [7:0]Mem[255:0];
12     always@(negedge CLK) begin
13         if (W_EN) begin
14             Mem[W_addr]<=W_data[31:24];
15             Mem[W_addr+1]<=W_data[23:16];
16             Mem[W_addr+2]<=W_data[15:8];
17             Mem[W_addr+3]<=W_data[7:0];
18         end
19     end
20     always@(RD_EN or R_addr) begin
21         if (RD_EN) begin
22             R_data<={Mem[R_addr],Mem[R_addr+1],Mem[R_addr+2],Mem[R_addr+3]};
23         end
24     end

```

6: Data_Mem.v

指令存储器:

按字节编址, 使用大端存储, 并把需要测试的指令存到指令内存中

```

1  `timescale 1ns / 1ps
2  module Ins_Mem(
3      input  [31:0]R_addr,
4      input  R_EN,
5      output reg[31:0]Ins
6  );
7      reg [7:0]Mem[255:0];
8      initial begin //用测试的指令来初始化Mem
9          $readmemb("D:/college/Computer_Lab/input.txt",Mem);
10     end
11     always@(R_addr or R_EN) begin
12         if (R_EN) begin //大端存储
13             Ins<={Mem[R_addr],Mem[R_addr+1],Mem[R_addr+2],Mem[R_addr+3]};
14         end
15     end
16 endmodule

```

7: Ins_Mem.v

寄存器堆:

寄存器读为组合逻辑电路, 写为时序逻辑电路, 为与 PC 区分, 寄存器写在时钟下降沿执行

```

1  `timescale 1ns / 1ps
2  module Reg_file(
3      input W_EN,
4      input CLK,
5      input [4:0]W_reg,
6      input [31:0]W_data,
7      input [4:0]RD_reg1,
8      input [4:0]RD_reg2,
9      output [31:0]RD_data1,
10     output [31:0]RD_data2
11 );
12     reg[31:0] registers[31:0];
13     integer i;
14     initial begin
15         for (i=0;i<32;i=i+1)
16             registers[i]<=0;
17     end
18     assign RD_data1=registers[RD_reg1];
19     assign RD_data2=registers[RD_reg2];
20     always@(negedge CLK) begin
21         if (W_EN) begin
22             registers[W_reg]<=W_data;

```

```

23         end
24     end
25 endmodule

```

8: Reg_file.v

PC 程序计数器:

```

1  `timescale 1ns / 1ps
2  module PC(
3      input CLK,
4      input Reset,
5      input W_EN,
6      input [31:0]nextaddress,
7      output reg [31:0]PC
8  );
9
10     always@(posedge CLK) begin
11         if (Reset) PC<=0;
12         else if (W_EN)
13             PC<=nextaddress;
14     end
15 endmodule

```

9: PC.v

5.2 顶层模块的综合

根据构建的数据通路以及原理, 实例化上述各组件, 计算下地址, 实现顶层模块 CPU

```

1  `timescale 1ns / 1ps
2  module CPU(
3      input CLK,
4      input Reset,
5      output [31:0]PC,
6      output [31:0]newaddress,
7      output [31:0]Instruction,
8      output [31:0]Reg_S,
9      output [31:0]Reg_T,
10     output [31:0]ALU_OUT,
11     output [31:0]W_data
12 );
13 // 控制器所需变量
14 wire Zero;
15 wire Sign;
16 wire PC_Wre;
17 wire ALUSrcA;
18 wire ALUSrcB;
19 wire Data_Src;
20 wire Reg_Wre;
21 wire Ins_Mem_RD;
22 wire Mem_RD;
23 wire Mem_Wre;

```

```

24     wire Reg_Dst;
25     wire ExtOP;
26     wire [2:0]ALUOP;
27     wire [1:0]PC_Src;
28     //ALU所需变量
29     wire [31:0]ALUA;
30     wire [31:0]ALUB;
31     //存储器
32     wire [31:0]Mem_OUT;
33     //扩展立即数
34     wire [31:0]immediate_32;
35     //寄存器堆
36     wire [4:0]W_Reg;
37     wire [31:0]PC4=PC+4;
38     assign newaddress= (PC_Src==2'b00)? PC4
39                                     :(PC_Src==2'b01) ? PC4+(immediate_32<<2)
40                                     :{PC4[31:28],Instruction[25:0],2'b00};
41     //ALU B口来自data2还是立即数
42     mul_32 mul_32_1(Reg_T,immediate_32,ALUsrcB,ALUB);
43     //目的寄存器是rt还是rd
44     mul_5 mul_5(Instruction[20:16],Instruction[15:11],Reg_Dst,W_Reg);
45     //ALU A口来自data1还是sa
46     mul_32 mul_32_2(Reg_S,{27'b00000000000000000000000000000000,Instruction[10:6]},ALUsrcA,ALUA);
47     //寄存器写来自ALU还是存储器
48     mul_32 mul_32_3(ALU_OUT,Mem_OUT,Data_Src,W_data);
49     //16位立即数扩展到32位
50     Ext_16_to_32 newimmediate(Instruction[15:0],ExtOP,immediate_32);
51
52     ALU myALU(ALUA,ALUB,ALUOP,ALU_OUT,Zero,Sign);
53     Control
54         myControl(Zero,Sign,Instruction[31:26],Instruction[5:0],PC_Wre,ALUsrcA,ALUsrcB,Data_Src,Reg_W
55 Ins_Mem_RD,Mem_RD,Mem_Wre,Reg_Dst,ExtOP,ALUOP,PC_Src);
56     Ins_Mem myIns_Mem(PC,Ins_Mem_RD,Instruction);
57     PC myPC(CLK,Reset,PC_Wre,newaddress,PC);
58     Data_mem myData(CLK,Mem_RD,Mem_Wre,ALU_OUT,ALU_OUT,Reg_T,Mem_OUT);
59     Reg_file myReg(Reg_Wre,CLK,W_Reg,W_data,Instruction[25:21],Instruction[20:16],Reg_S,Reg_T);
60 endmodule

```

10: CPU.v

5.3 对 CPU 进行仿真测试

```

1  `timescale 1ns / 1ps
2  module CPU_tb();
3      reg CLK;
4      reg Reset;
5      wire [31:0]PC;
6      wire [31:0]newaddress;
7      wire [31:0]Instruction;
8      wire [31:0]Reg_S;
9      wire [31:0]Reg_T;
10     wire [31:0]ALU_OUT;
11     wire [31:0]W_data;

```

```

12 //CPU mycpu(CLK,Reset,PC,newaddress,Instruction,Reg_S,Reg_T,ALU_OUT,W_data);
13 CPU mycpu(
14     .CLK(CLK),
15     .Reset(Reset),
16     .PC(PC),
17     .newaddress(newaddress),
18     .Instruction(Instruction),
19     .Reg_S(Reg_S),
20     .Reg_T(Reg_T),
21     .ALU_OUT(ALU_OUT),
22     .W_data(W_data)
23 );
24 integer i;
25 initial begin
26     Reset=1;CLK=0;
27     #50;
28     CLK=1;
29     #50;
30     Reset=0;CLK=0;
31     for (i=0;i<100;i=i+1)
32     begin
33         #50;
34         CLK=!CLK;
35     end
36 end
37 endmodule

```

11: CPU_tb.v

其中指令存储在 Ins_Mem 中, 指令内容与对应 mips 机器码为

```

addiu $1 $0 8 00100100 00000001 00000000 00001000
ori $2 $0 2 00110100 00000010 00000000 00000010
add $3 $2 $1 00000000 01000001 00011000 00100000
sub $5 $3 $2 00000000 01100010 00101000 00100010
and $4 $5 $2 00000000 10100010 00100000 00100100
or $8 $4 $2 00000000 10000010 01000000 00100101
sll $8 $8 1 00000000 00001000 01000000 01000000
bne $8 $1 -2 00010101 00000001 11111111 11111110
slti $6 $2 4 00101000 01000110 00000000 00000100
slti $7 $6 0 00101000 11000111 00000000 00000000
addiu $7 $7 8 00100100 11100111 00000000 00001000
beq $7 $1 -2 00010000 11100001 11111111 11111110
sw $2 4($1) 10101100 00100010 00000000 00000100

```

```

lw $9 4($1) 10001100 00101001 00000000 00000100
addiu $10 $0 -2 00100100 00001010 11111111 11111110
addiu $10 $10 1 00100101 01001010 00000000 00000001

blez $10 -2 00011001 01000000 11111111 11111110

andi $11 $2 2 00110000 01001011 00000000 00000010

j 0x0000004C 00001000 00000000 00000000 00010011

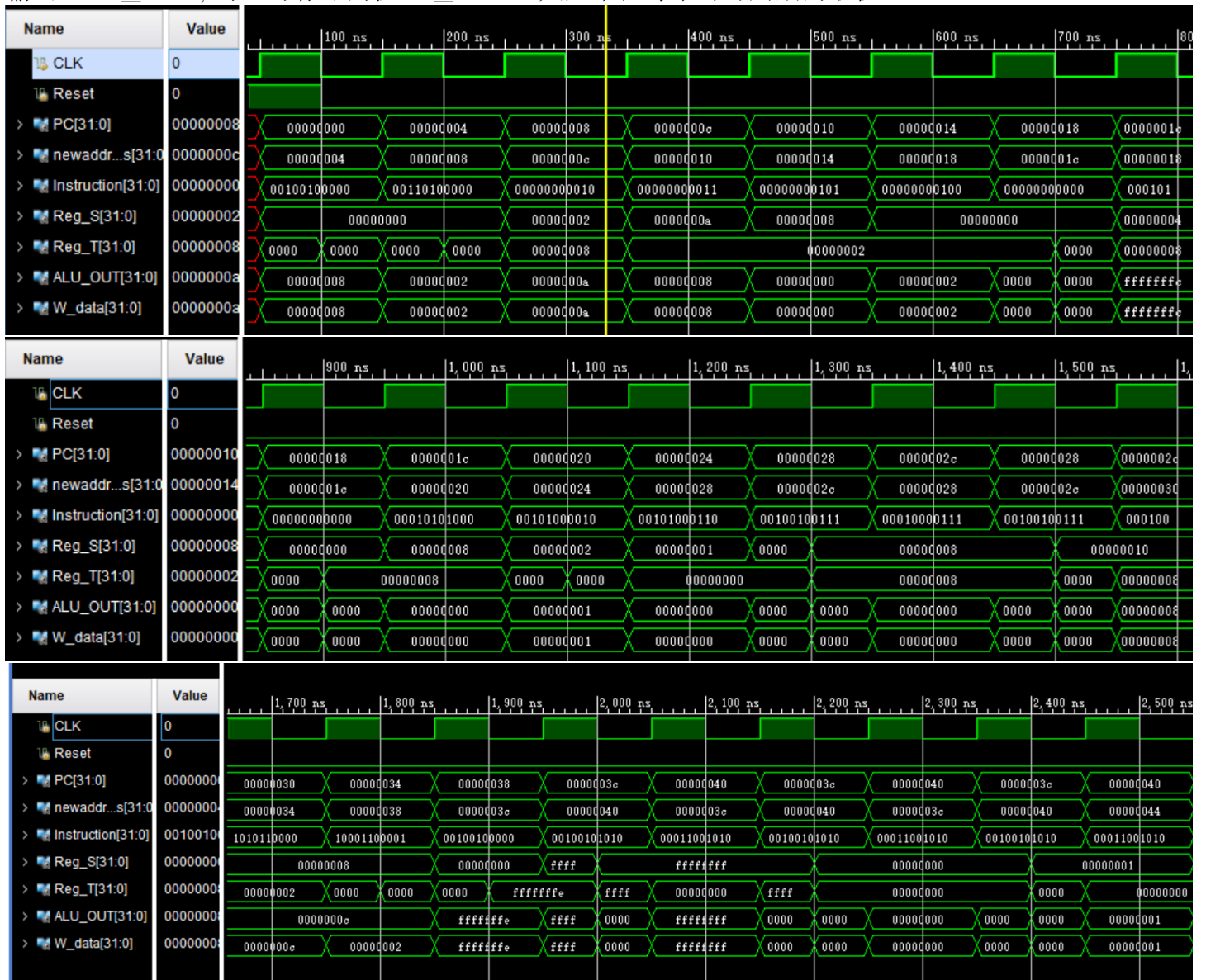
or $8 $4 $2 00000000 10000010 01000000 00100101

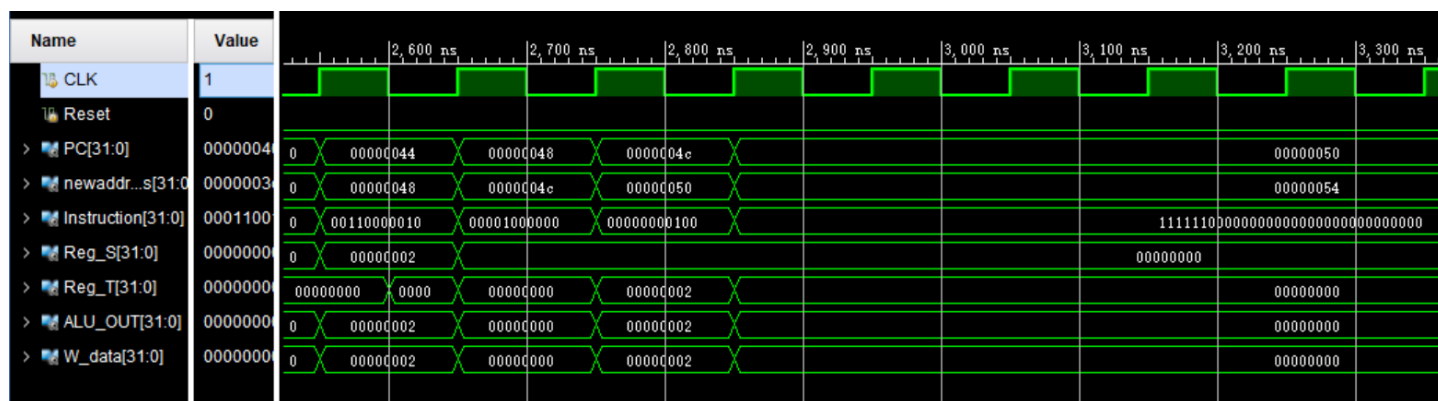
halt 11111100 00000000 00000000 00000000

```

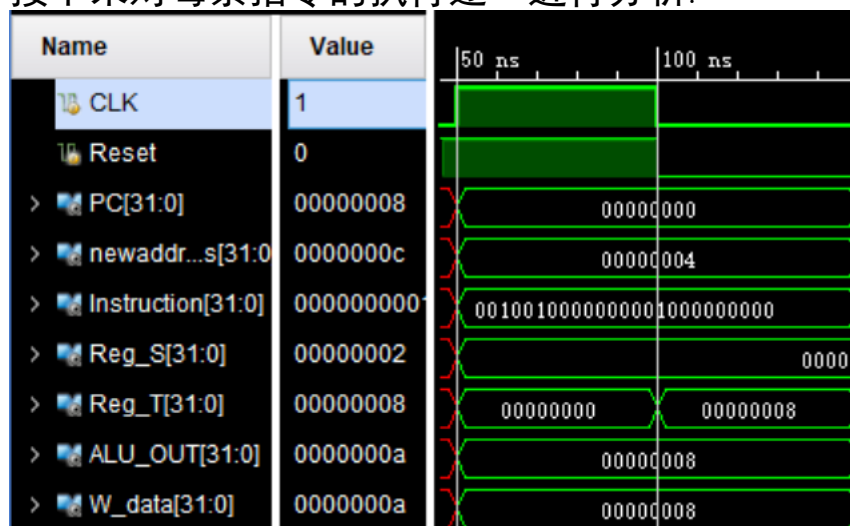
5.4 得到波形图以及分析

CPU 执行完所有指令得到的全部波形图如下, 其中波形图记录了时钟 CLK,PC 复位信号 Reset, 当前 PC, 下地址 newaddress, 指令 Instruction, 指令 rs、rt 字段对应寄存器中的值 Reg_S,Reg_T,ALU 的输出 ALU_OUT, 写入寄存器的值 W_data 总共九个在每个时钟周期的取值。

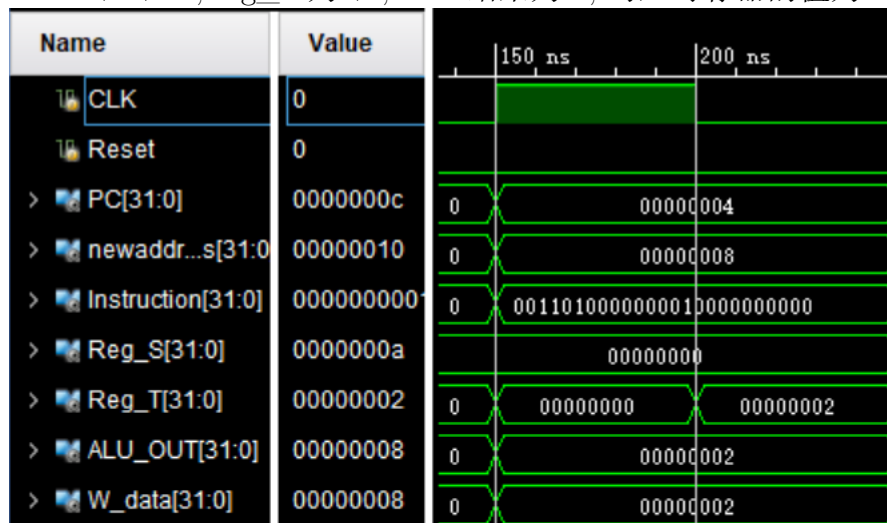




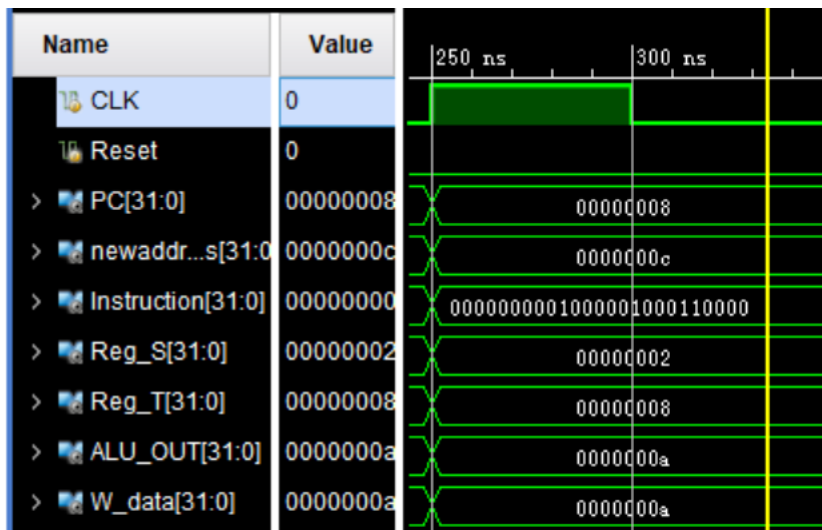
接下来对每条指令的执行逐一进行分析:



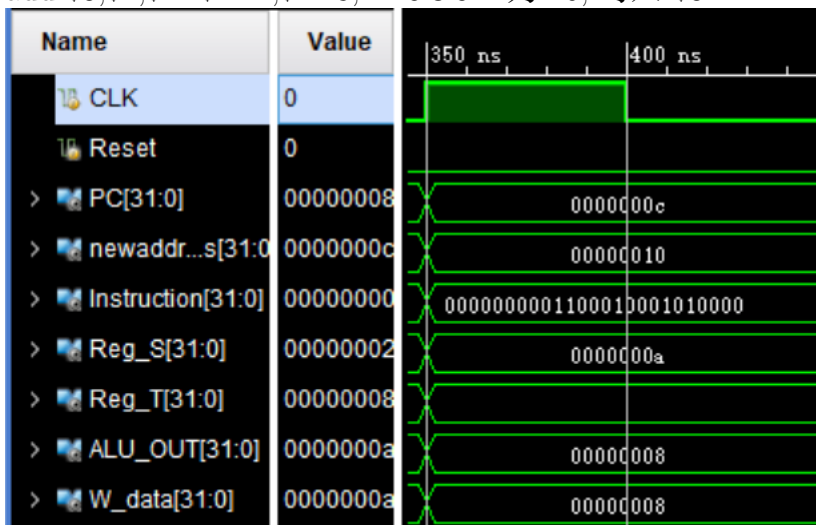
addiu \$1 \$0 8,Reg_S 为 \$1,ALU 结果为 8, 写入寄存器的值为 8, 在时钟下降沿写入 \$1



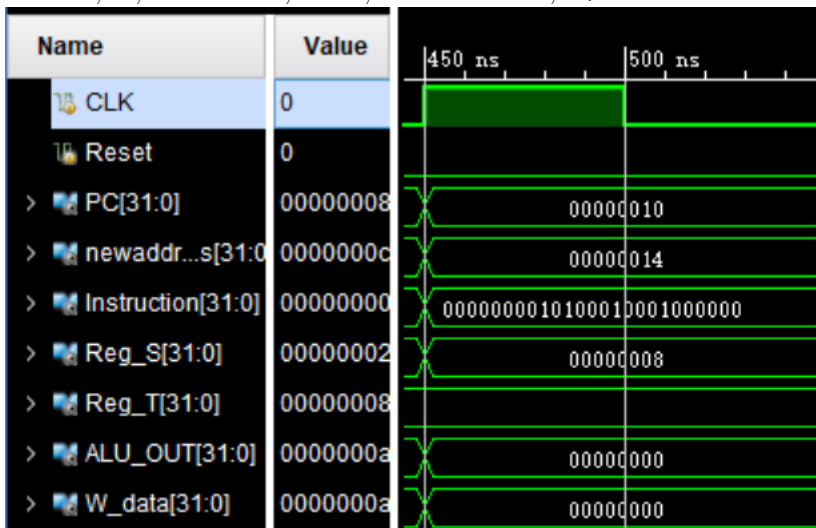
ori \$2,\$0,2: ALU 输出为 2, 在时钟下降沿写入 \$2



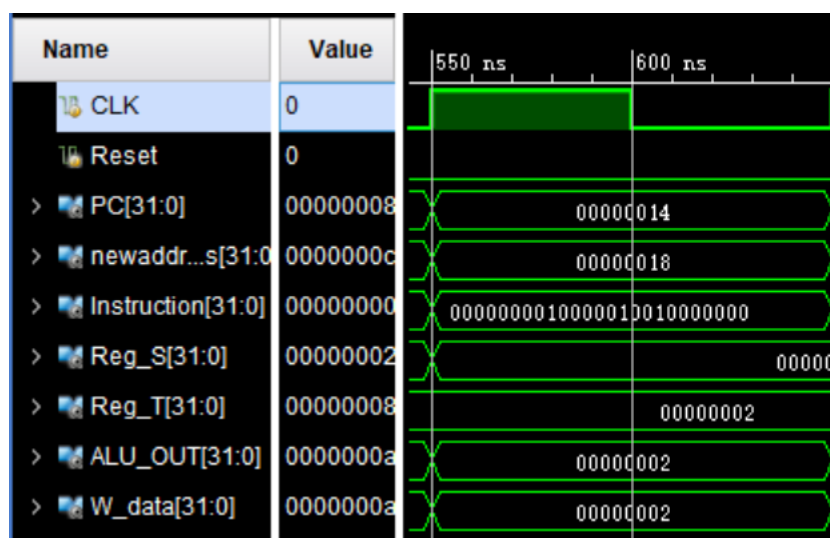
add \$3,\$2,\$1: \$2=2,\$1=8,ALUOUT 为 10, 写入 \$3



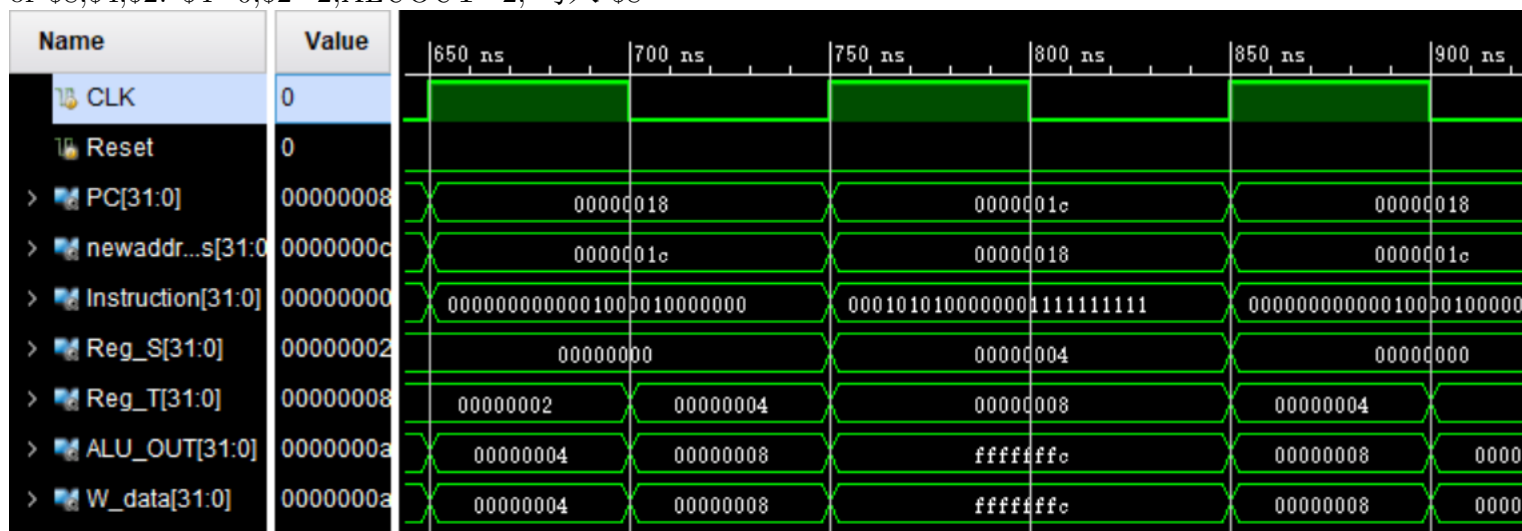
sub \$5,\$3,\$2: \$3=10,\$2=2,ALUOUT=8, 写入 \$5



and \$4,\$5,\$2: \$5=8,\$2=2,ALUOUT 为 0, 写入 \$4



or \$8,\$4,\$2: \$4=0,\$2=2,ALUOUT=2, 写入 \$8



0x18 sll \$8,\$8,1

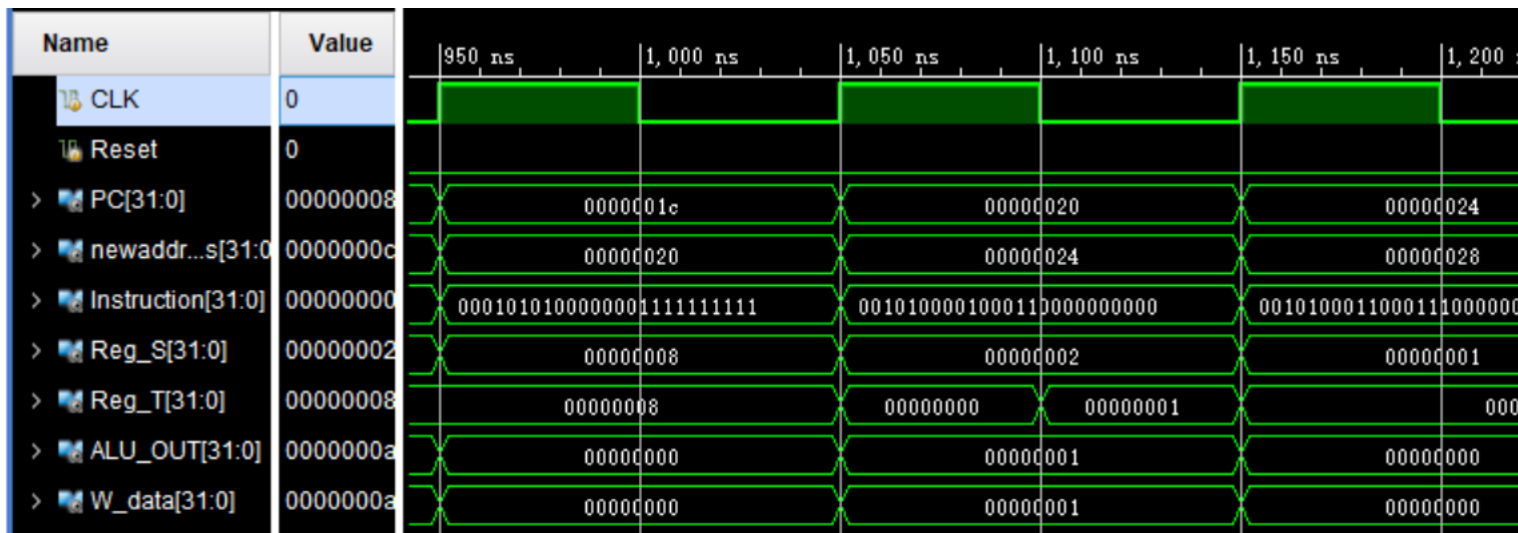
0x1c bne \$8,\$1,-2 (, 转 18)

0x18 sll \$8,\$8,1

0x1c bne \$8,\$1,-2 (, 转 18)

\$8 左移一位后进行分支判断, 第一次分支判断执行分支,\$8 继续移位后进行第二次判断, 第二次不执行分支

执行完上述指令后,\$8=8

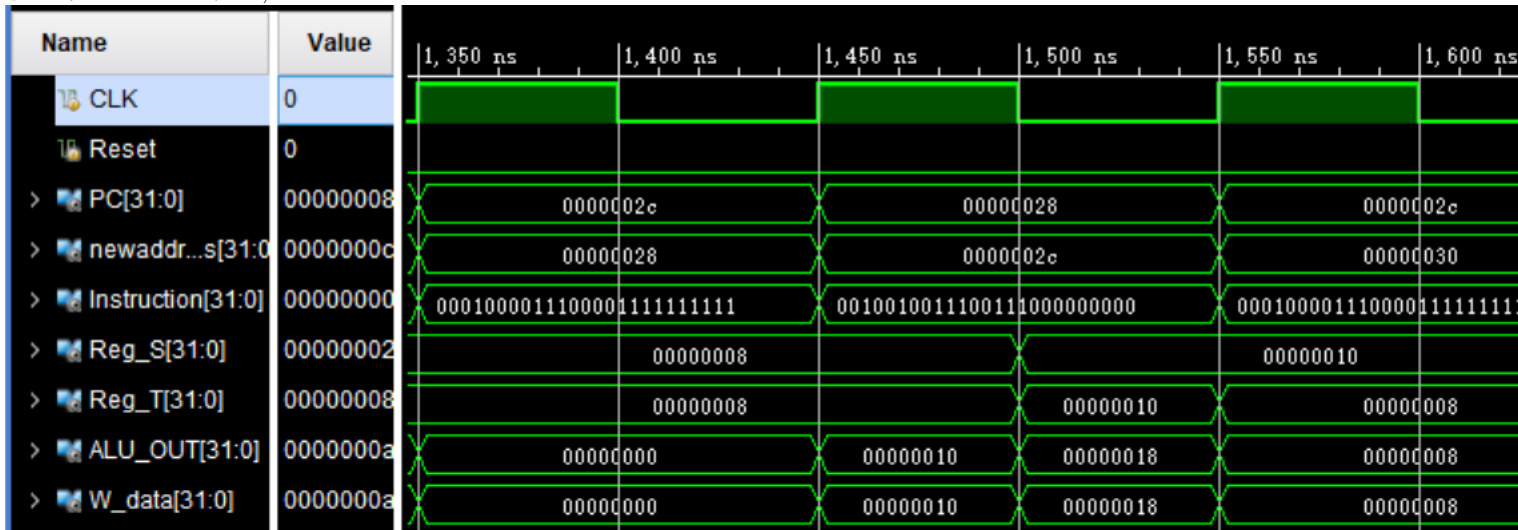


slti \$6,\$2,4

slti \$7,\$6,0

addiu \$7,\$7,8

执行完上述指令后,\$6=1 \$7=8

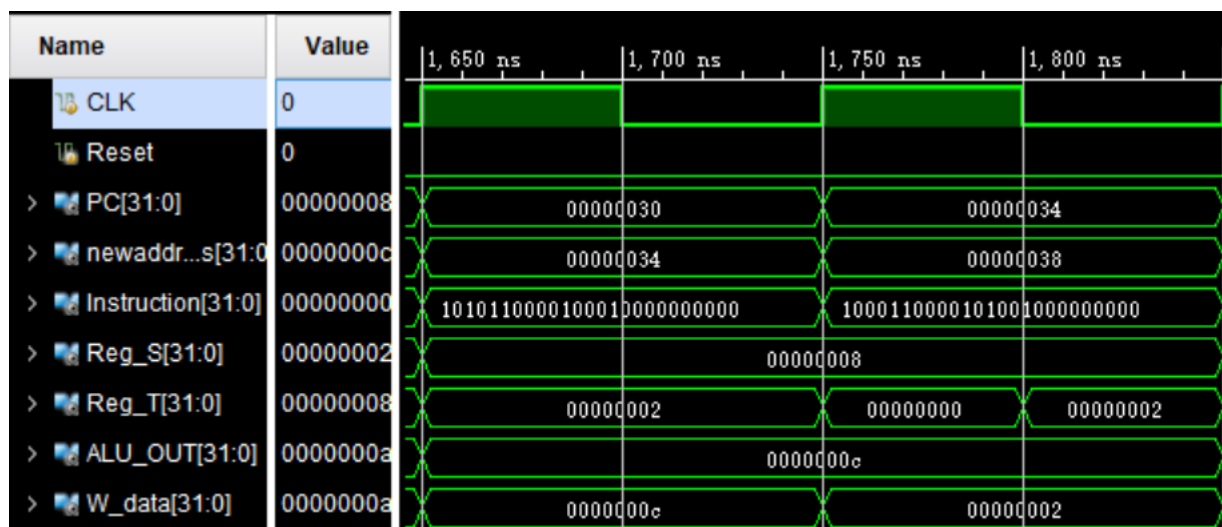


0x2c beq \$7,\$1,-2 (=, 转 28)

0x28 addiu \$7,\$7,8

0x2c beq \$7,\$1,-2 (=, 转 28)

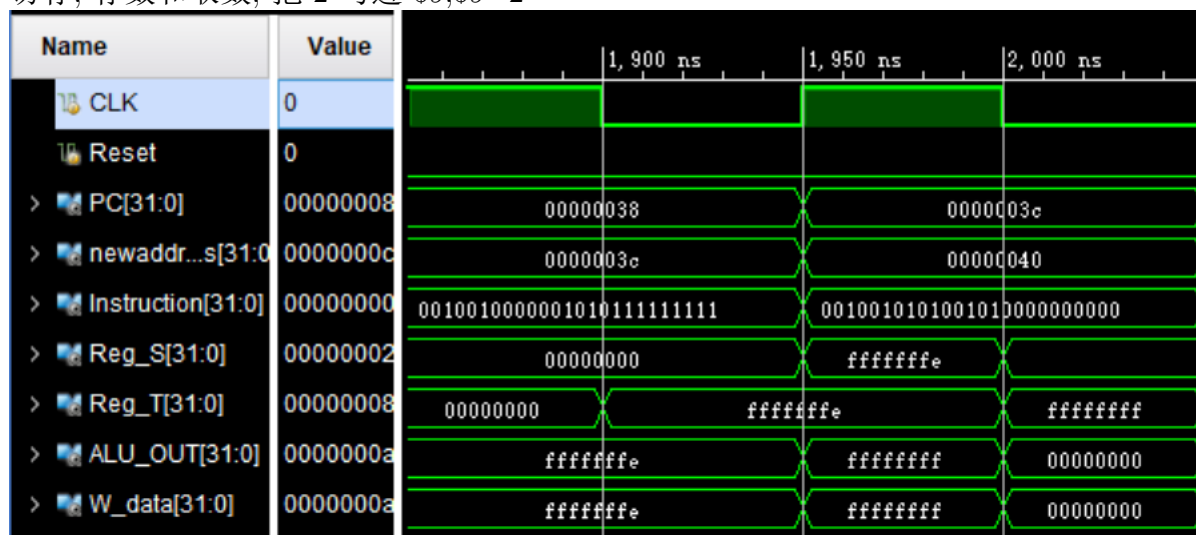
第一次分支指令发生, 第二次不发生, 执行完后 \$7=16



sw \$2,4(\$1)

lw \$9,4(\$1)

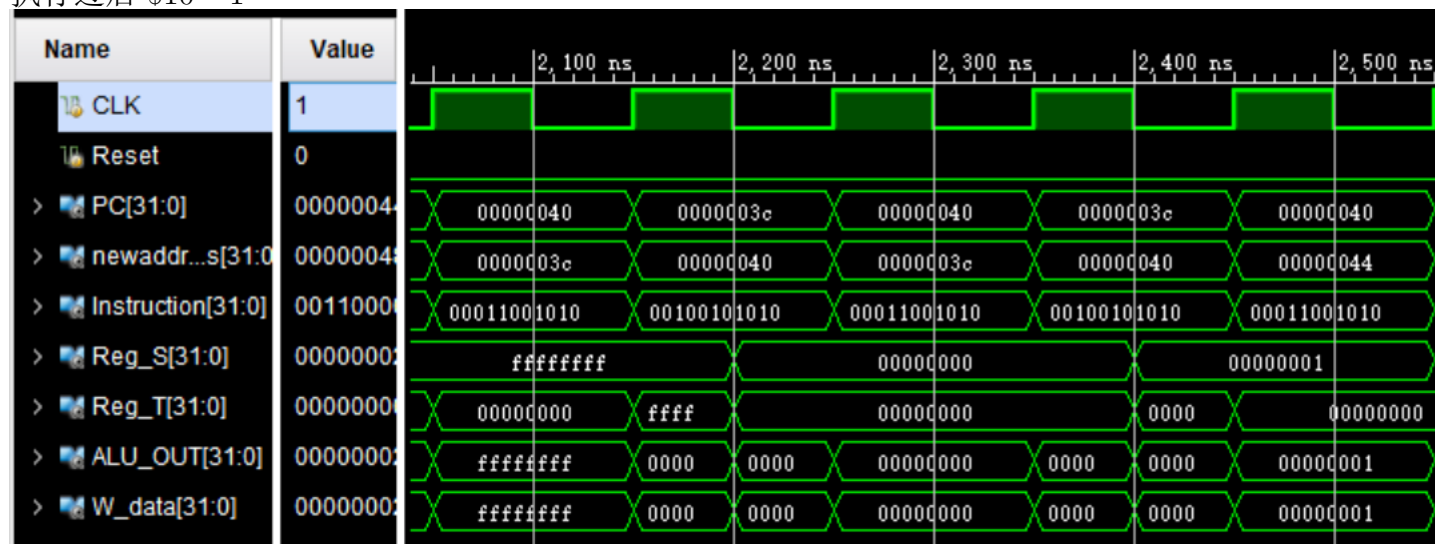
访存, 存数和取数, 把 2 写进 \$9,\$9=2



addiu \$10,\$0,-2

addiu \$10,\$10,1

执行过后 \$10=-1



0x40 blez \$10,-2

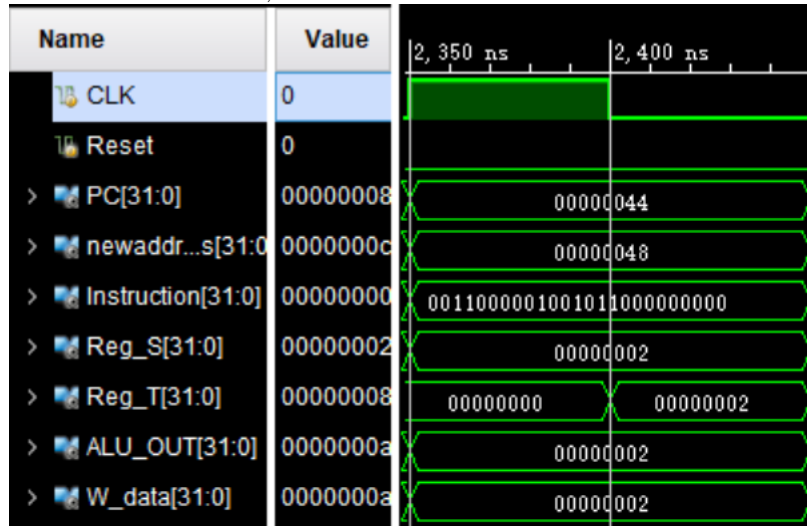
0x3c addiu \$10,\$10,1

0x40 blez \$10,-2

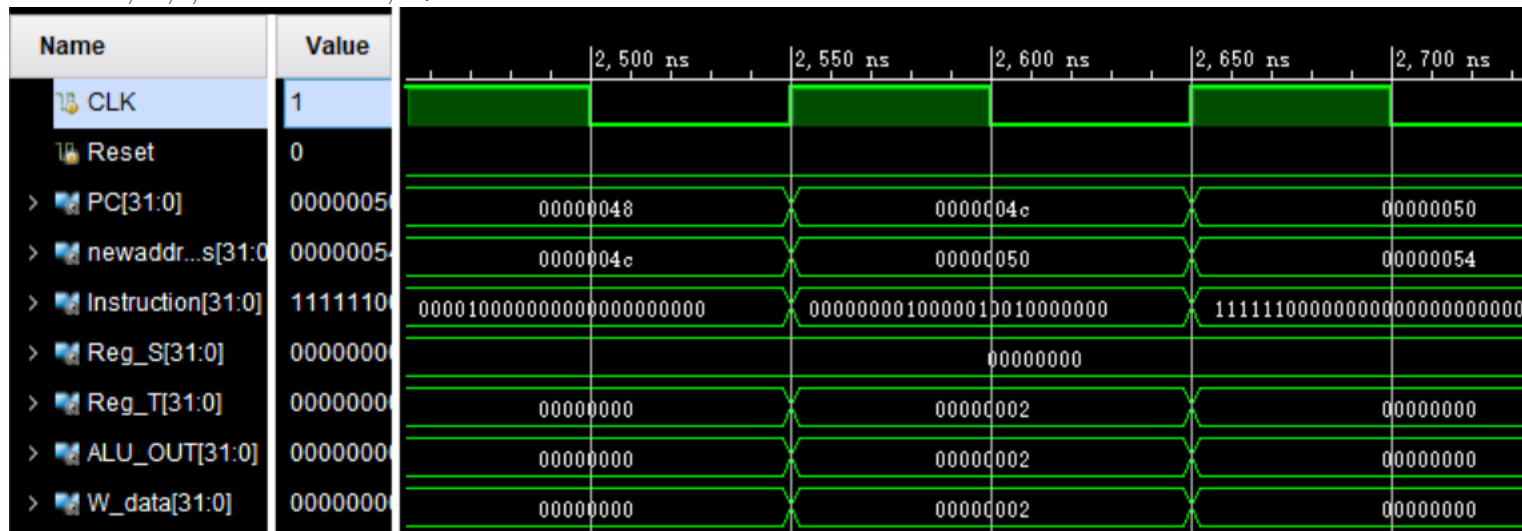
0x3c addiu \$10,\$10,1

0x40 blez \$10,-2

前两次分支均发生,\$10=1



andi \$11,\$2,2, ALUOUT=2, 写入 \$11



0x48 j 0x0000004C

0x4c or \$8,\$4,\$2

0x50 halt

无条件跳转: 无条件跳转到 0x4C

接着执行 or 指令,\$4 为 0,\$2 为 2, 或运算得到结果为 2

Halt 指令后, 时钟上升沿到来时不更新 PC

5.5 用 basys3 板来运行设计的 CPU

5.6 设计思路

- 实现 CPU 在 basys3 板上运行, 需要两个时钟: 一个是 basys3 板本身的时钟 (对应管脚为 W5), 默认频率为 100MHZ; 另一个是 CPU 的时钟, 由我们手动进行按键产生 (basys3 板上的 BTNR 按键产生脉冲)。
- 每个按键周期, 4 个数码管都必须亮一次, 因此我们需要将 basys3 的时钟分频, 使得 4 个数码管轮流点亮 (造成在视觉上”同时”点亮的效果)
- 由于 basys3 采用机械按键, 极其不稳定, 在按下按键的瞬间会产生短些间内频繁高低变化的电平, 产生抖动, 影响程序运行。因此需要添加按键消抖模块, 避免抖动对 CPU 运行的影响。

5.7 实现过程

分频器, 输出数码管显示的控制信号, 使四个数码管轮流显示:

```
1  `timescale 1ns / 1ps
2  module frequency_divider(
3      input CLK,
4      output reg[3:0] AN
5  );
6      parameter T1MS= 100000;
7      reg [21:0] counter;
8      initial begin
9          counter<= 0;
10         AN<= 4'b0111;
11     end
12     always@(posedge CLK) begin
13         counter<=counter+1;
14         if (counter==T1MS) begin
15             counter<=0;
16             case (AN)
17                 4'b0111: AN<=4'b1110;
18                 4'b1110: AN<=4'b1101;
19                 4'b1101: AN<=4'b1011;
20                 4'b1011: AN<=4'b0111;
21                 4'b0000: AN<=4'b0111;
22             endcase
23         end
24     end
25 endmodule
```

12: frequency_divider.v

按键消抖模块, 只有按键在高电平/低电平稳定一段时间之后, CPU 的时钟 (myCLK) 才自动更新, 避免了抖动带来的影响

```

1  `timescale 1ns / 1ps
2  module button_debounce(
3      input CLK,
4      input button,
5      output out
6  );
7      parameter MAX=5000;
8      reg [15:0] count_low;
9      reg [15:0] count_high;
10     reg count_out;
11     always@(posedge CLK) begin
12         count_low<= button?0:count_low+1;
13         count_high<= button?count_high+1:0;
14         if (count_low==MAX) count_out<=0;
15         else if (count_high==MAX) count_out<=1;
16     end
17     assign out=!count_out;
18 endmodule

```

13: button_debounce.v

十六进制 0-'F' 到七段数码管之间的转换, 注意到 basys3 采用共阳极

```

1  `timescale 1ns / 1ps
2  module display_7seg(
3      input [3:0] data,
4      output reg[7:0] display
5  );
6      always@(data) begin
7          case (data)
8              4'b0000 : display = 8'b1100_0000;  //0; '0'-亮灯, '1'-熄灯
9              4'b0001 : display = 8'b1111_1001;  //1
10             4'b0010 : display = 8'b1010_0100;  //2
11             4'b0011 : display = 8'b1011_0000;  //3
12             4'b0100 : display = 8'b1001_1001;  //4
13             4'b0101 : display = 8'b1001_0010;  //5
14             4'b0110 : display = 8'b1000_0010;  //6
15             4'b0111 : display = 8'b1101_1000;  //7
16             4'b1000 : display = 8'b1000_0000;  //8
17             4'b1001 : display = 8'b1001_0000;  //9
18             4'b1010 : display = 8'b1000_1000;  //A
19             4'b1011 : display = 8'b1000_0011;  //b
20             4'b1100 : display = 8'b1100_0110;  //C
21             4'b1101 : display = 8'b1010_0001;  //d
22             4'b1110 : display = 8'b1000_0110;  //E
23             4'b1111 : display = 8'b1000_1110;  //F
24             default : display = 8'b0000_0000;  //不亮
25         endcase
26     end
27 endmodule

```

14: display_7seg.v

顶层综合模块, 由消抖后的按键产生 CPU 的时钟 (myCLK), 并根据实验要求, 给不同的输入 SW_15、SW_14, 显示对应的数据, 并分配给数码管显示;

注意到指令地址的范围为 0-255, 存储器地址范围为 0-255, 因此只需要两位十六进制, 即两个数码管即足够显示一个数据

```
1  `timescale 1ns / 1ps
2  module basys3(
3      input CLK,
4      input [1:0]SW,
5      input Reset,
6      input button,
7      output [3:0]AN,
8      output [7:0]display
9  );
10     wire [31:0]PC;
11     wire [31:0]newaddress;
12     wire [31:0]Instruction;
13     wire [31:0]Reg_S;
14     wire [31:0]Reg_T;
15     wire [31:0]ALUOUT;
16     wire [31:0]W_data;
17     wire myCLK;
18     reg [3:0]data;
19     button_debounce mybutton(CLK,button,myCLK);
20     CPU mycpu(myCLK,Reset,PC,newaddress,Instruction,Reg_S,Reg_T,ALUOUT,W_data);
21     display_7seg myseg(data,display);
22     frequency_divider my(CLK,AN);
23     always@(myCLK)begin
24         case(AN)
25             4'b1110:    begin
26                 case(SW)
27                     2'b00:data<=newaddress[3:0];
28                     2'b01:data<=Reg_S[3:0];
29                     2'b10:data<=Reg_T[3:0];
30                     2'b11:data<=W_data[3:0];
31                 endcase
32             end
33             4'b1101:    begin
34                 case(SW)
35                     2'b00:data<=newaddress[7:4];
36                     2'b01:data<=Reg_S[7:4];
37                     2'b10:data<=Reg_T[7:4];
38                     2'b11:data<=W_data[7:4];
39                 endcase
40             end
41             4'b1011:    begin
42                 case(SW)
43                     2'b00:data<=PC[3:0];
44                     2'b01:data<=Instruction[24:21];
45                     2'b10:data<=Instruction[19:16];
46                     2'b11:data<=ALUOUT[3:0];
```

```

47         endcase
48     end
49     4'b0111 : begin
50         case(SW)
51             2'b00:data<=PC[7:4];
52             2'b01:data<={3'b000,Instruction[25]};
53             2'b10:data<={3'b000,Instruction[20]};
54             2'b11:data<=ALUOUT[7:4];
55         endcase
56     end
57 endcase
58 end
59 endmodule

```

15: basys3.v

引脚分配:

位选 SW[1:0] 分配到 basys3 的开关 SW_15 和 SW_14

button 分配给 basys3 上的脉冲 BTNR, 手动控制 CPU 时钟

七段数码管输出 display[7:0] 从左至右分配给 basys3 上的 DP、CG、CF、CE、CD、CC、CB、CA

数码管位选信号 AN[3:0] 分配给 basys3 上的数码管位选 AN3-AN0

清零信号 Reset 分配给 basys3 上的开关 SW_0

时钟 CLK 分配到 basys3 上的时钟引脚 W5

引脚分配文件如下:

```

1 set_property IOSTANDARD LVCMOS33 [get_ports {AN[3]}]
2 set_property IOSTANDARD LVCMOS33 [get_ports {AN[2]}]
3 set_property IOSTANDARD LVCMOS33 [get_ports {AN[1]}]
4 set_property IOSTANDARD LVCMOS33 [get_ports {AN[0]}]
5 set_property PACKAGE_PIN U2 [get_ports {AN[0]}]
6 set_property PACKAGE_PIN U4 [get_ports {AN[1]}]
7 set_property PACKAGE_PIN V4 [get_ports {AN[2]}]
8 set_property PACKAGE_PIN W4 [get_ports {AN[3]}]
9 set_property IOSTANDARD LVCMOS33 [get_ports {SW[1]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {SW[0]}]
11 set_property PACKAGE_PIN R2 [get_ports {SW[1]}]
12 set_property PACKAGE_PIN T1 [get_ports {SW[0]}]
13 set_property IOSTANDARD LVCMOS33 [get_ports button]
14 set_property PACKAGE_PIN T17 [get_ports button]
15 set_property IOSTANDARD LVCMOS33 [get_ports CLK]
16 set_property PACKAGE_PIN W5 [get_ports CLK]
17 set_property IOSTANDARD LVCMOS33 [get_ports Reset]
18 set_property PACKAGE_PIN V17 [get_ports Reset]
19 set_property IOSTANDARD LVCMOS33 [get_ports {display[7]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {display[6]}]
21 set_property IOSTANDARD LVCMOS33 [get_ports {display[5]}]
22 set_property IOSTANDARD LVCMOS33 [get_ports {display[4]}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {display[3]}]
24 set_property IOSTANDARD LVCMOS33 [get_ports {display[2]}]

```

```

25 set_property IOSTANDARD LVCMOS33 [get_ports {display[1]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {display[0]}]
27 set_property PACKAGE_PIN V7 [get_ports {display[7]}]
28 set_property PACKAGE_PIN U7 [get_ports {display[6]}]
29 set_property PACKAGE_PIN V5 [get_ports {display[5]}]
30 set_property PACKAGE_PIN U5 [get_ports {display[4]}]
31 set_property PACKAGE_PIN V8 [get_ports {display[3]}]
32 set_property PACKAGE_PIN U8 [get_ports {display[2]}]
33 set_property PACKAGE_PIN W6 [get_ports {display[1]}]
34 set_property PACKAGE_PIN W7 [get_ports {display[0]}]

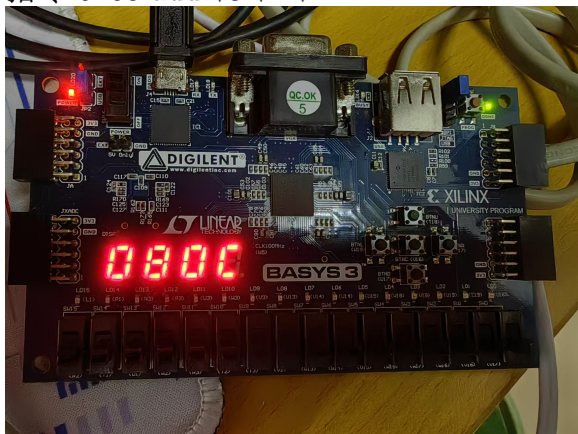
```

16: constrain.xdc

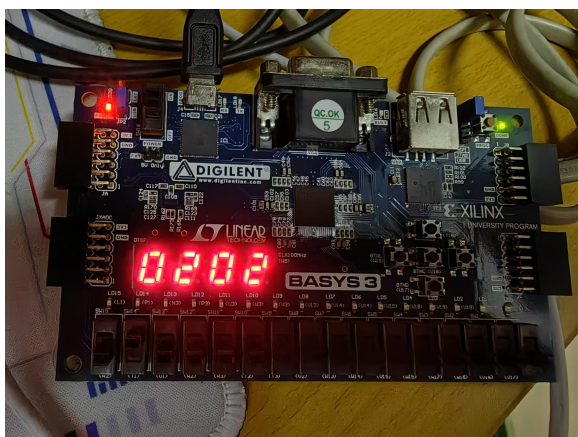
5.8 烧板结果

烧板到 CPU 后, 由 basys3 执行所给程序, 选取了部分结果进行展示, 如下:

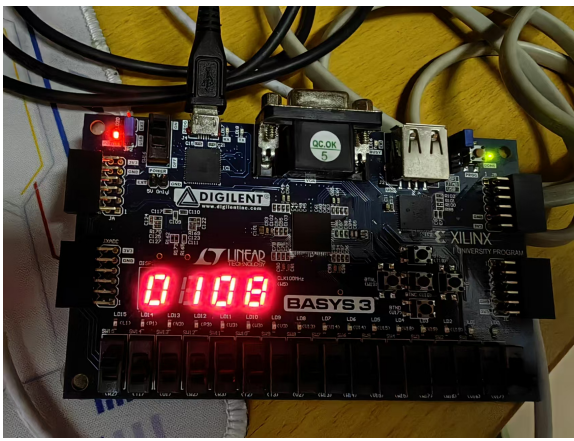
- 指令: 0x08 add \$3 \$2 \$1



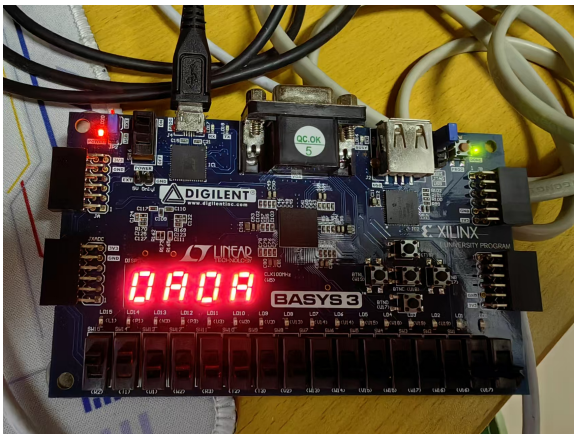
开关置位 00, 当前 PC 为 08, 下地址为 0C



开关置位 01, \$2=2

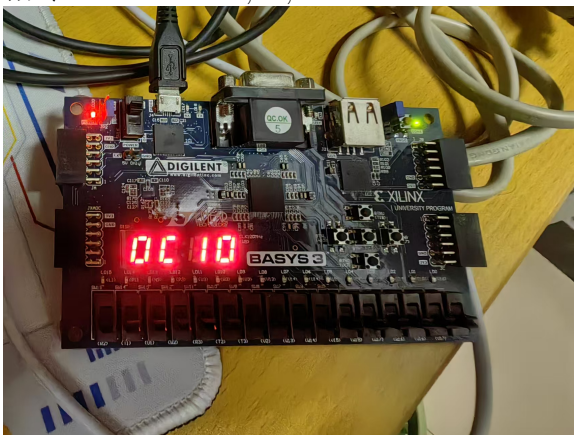


开关置位 10,\$1=8

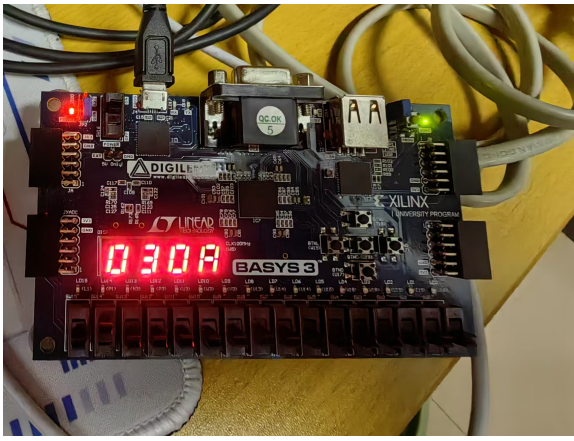


开关置位 11,ALUOUT=10(16 进制为 0A)

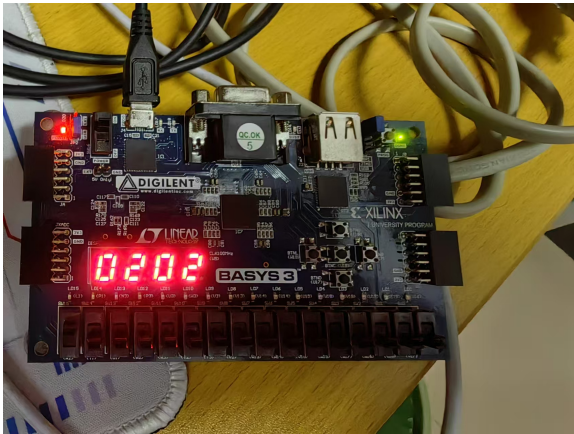
- 指令:0x0C sub \$5,\$3,\$2



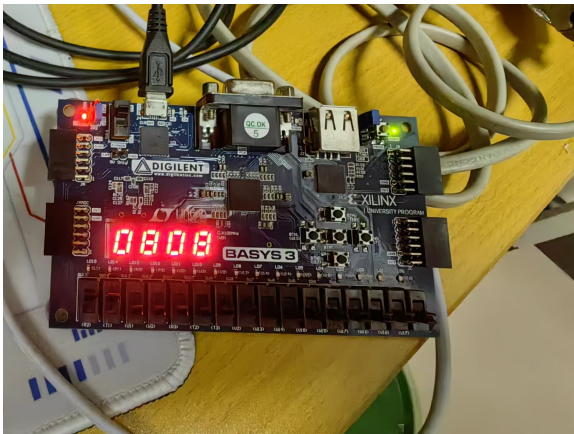
开关置位 00,PC 为 0x0C, 下地址为 0x10



开关置位 01,\$3=10(0x0A)

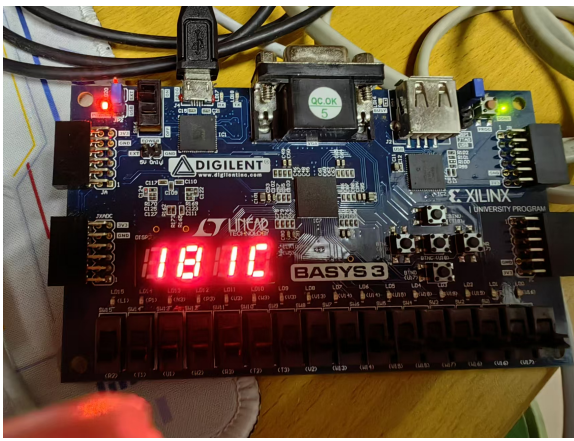


开关置位 10,\$2=2

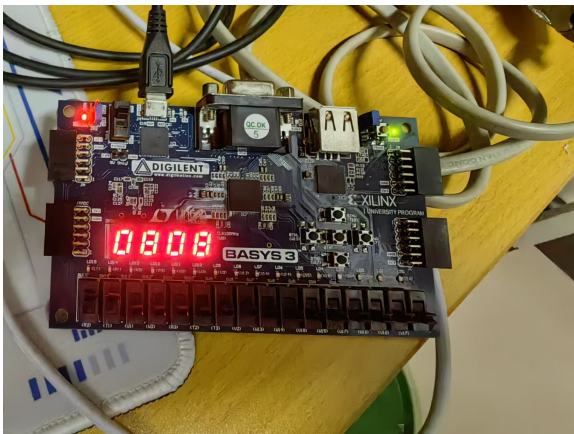


开关置位 11,ALUOUT=10-2=8

- 指令:0x18 sll \$8,\$8,1

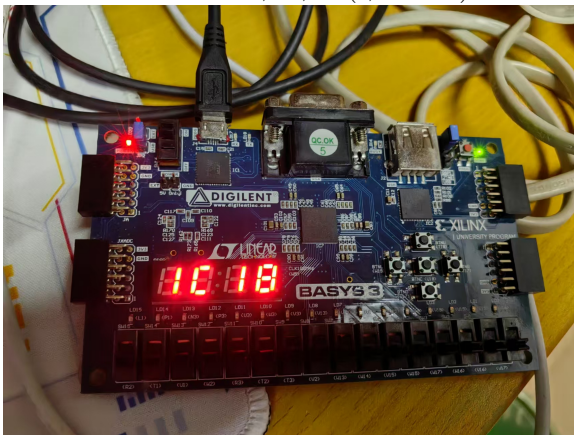


开关置位 00, 当前地址为 0x18, 下地址为 0x1C



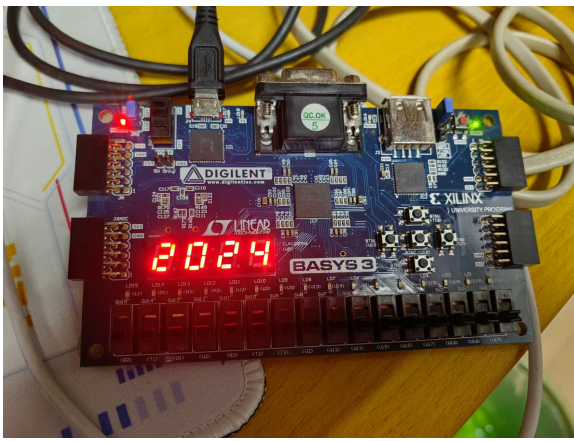
开关置位 11, $ALUOUT = 4 \ll 1 = 8$

- 指令: 0x1C bne \$8,\$1,-2 (, 转 18)

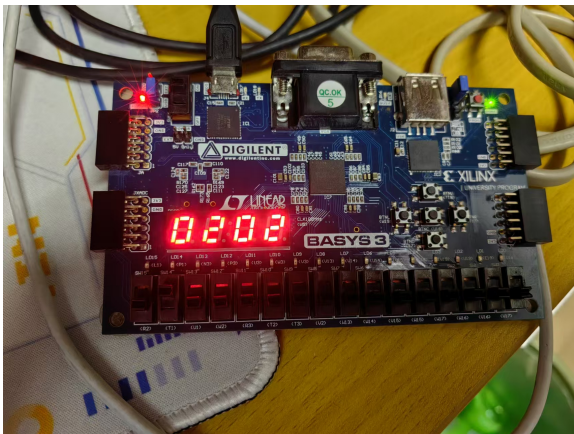


开关置位 00, 分支发生, 下地址为 0x18

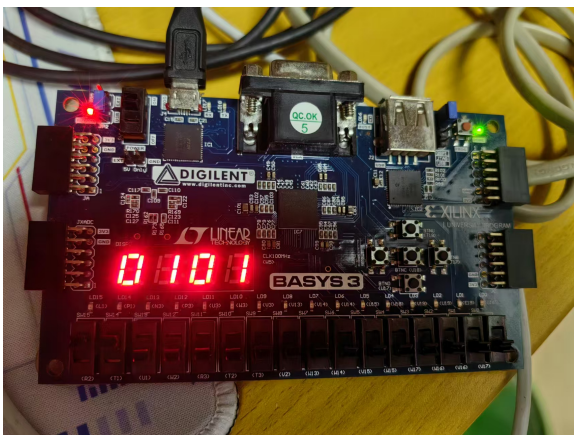
- 指令: 0x20 slti \$6,\$2,4



开关置位 00, 当前地址 0x20, 下地址 0x24

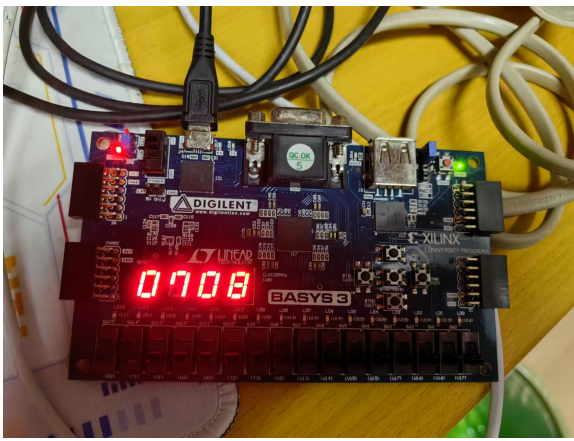


开关置位 01, \$2=2

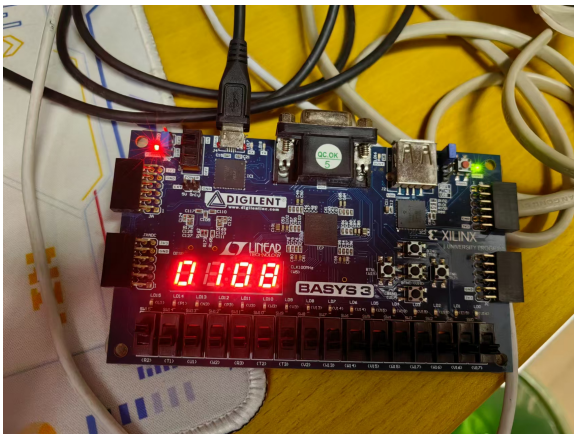


开关置位 11, $2 < 4$, ALUOUT 和写入 \$6 的值均为 1

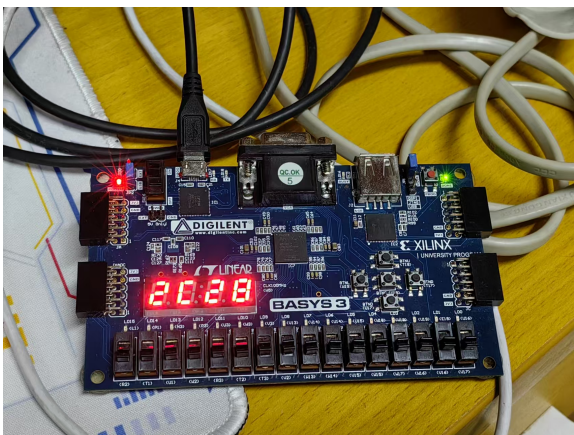
- 指令: 0x2C beq \$7,\$1,-2 (=, 转 28)



开关置位 01,\$7=8

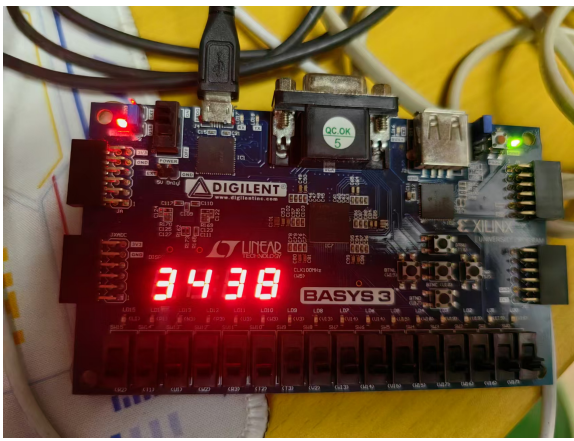


开关置位 10,\$1=8=\$7, 因此分支发生

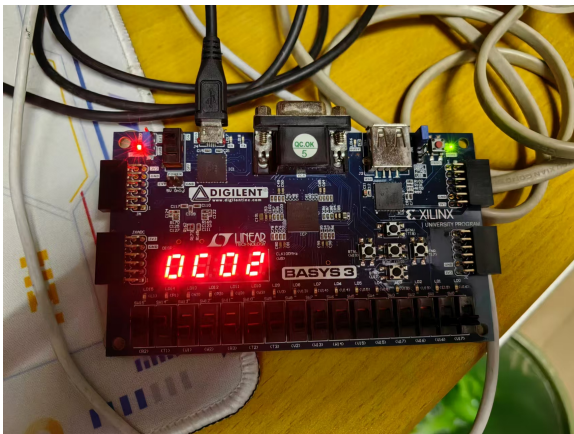


开关置位 00, 分支发生, 下地址为 0x28

- 指令: 0x34 lw \$9,4(\$1)

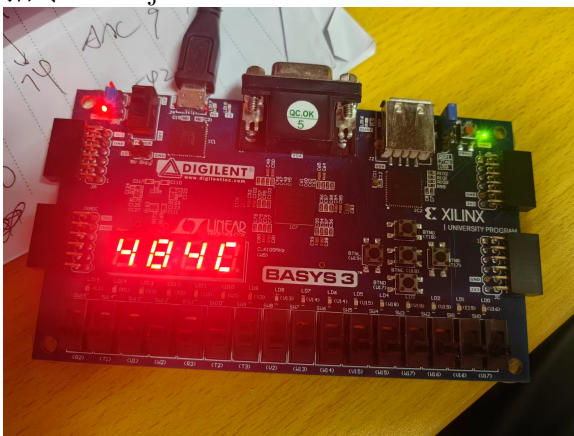


开关置位 00, 下地址为 0x38



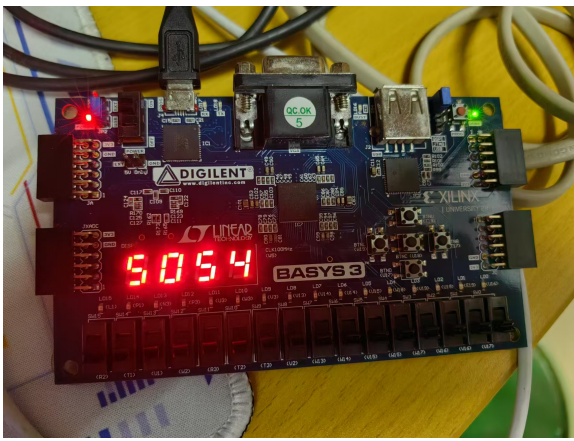
开关置位 11, 从存储器读出并加载到 \$9 的值为 2

- 指令: 0x48 j 0x0000004C



开关置位 00, 无条件跳转到 0x4C

- 指令: 0x50 halt



开关置位 00,PC 为 50, 且 halt 指令, 时钟上升沿到来时不写 PC,PC 一直为 50

6 实验心得

- 在使用 vivado 编写代码前, 需要先做好充分的准备: 根据单周期 CPU 的数据通路, 提前总结好各模块的功能 (有什么 input、有什么 output), 以及通过表格的形式整理各个控制信号的取值、功能, 以及每条指令单独对应什么控制信号。在做好这些充足的准备时, 编写各模块的代码才会更加得心应手。
- 由于 vivado 不像我们平时写 c、c++、py 等语言, 没有精确到行的报错。因此 debug 是极其重大的挑战。对于极其重要的模块, 可以单独拿出来编写仿真文件, 通过波形图来 debug, 但如果对所有文件都这么做则很浪费时间。

因此我首先会仔细检查每个变量有没有漏了分配宽度 (我就因为我有文件中的 W_data 忘记分配了 [31:0] 的宽度, 导致波形图”一片红”)

而对于控制信号的 debug, 由于最终程序的 output 较少, 不好发现 bug, 我选择另写一个 CPU 程序, 将所有控制信号设为 output

```

1    `timescale 1ns / 1ps
2    module CPU(
3        input CLK,
4        input Reset,
5        output [31:0] PC,
6        output [31:0] newaddress,
7        output [31:0] Instruction,
8        output [31:0] Reg_S,
9        output [31:0] Reg_T,
10       output [31:0] ALU_OUT,
11       output [31:0] W_data,
12       output Zero,
13       output Sign,
14       output PC_Wre,

```

```

15     output ALUsrcA,
16     output ALUsrcB,
17     output Data_Src,
18     output Reg_Wre,
19     output Ins_Mem_RD,
20     output Mem_RD,
21     output Mem_Wre,
22     output Reg_Dst,
23     output ExtOP,
24     output [2:0]ALUOP,
25     output [1:0]PC_Src,
26     output [4:0]W_Reg
27 );

```

17: control_debug.v

这样便能通过波形图仔细的找出了我有部分控制信号的逻辑出现了错误, 导致结果不正确

- 在编写仿真模块时, 已有的参考代码中, 在产生周期性的时钟信号时, 用的是 forever:

```

1     forever #50 begin //产生时钟信号,周期为 50s
2         CLK= !CLK;
3     end

```

我用这个进行仿真时, 由于指令极其有限,forever 进行仿真产生了很多冗余周期, 产生了大部分无效波形图 (PC=0x50 之后指令不在执行), 波形图不好看清楚, 于是便改成了

```

1     for (i=0;i<100;i=i+1)
2     begin
3         #50;
4         CLK=!CLK;
5     end

```

这样使得仿真在 PC=0x50 之后的几个周期后即结束

- 分配引脚有两种方法, 编写 xdc 文件和手动分配引脚。我最开始的分配方式是对着现有的参考 xdc 引脚分配文件, 对应自己的程序进行修改, 编写自己的 xdc 文件。但是尝试烧板的时候总会报错 fail。于是我运用了手动分配引脚的功能, 而不是编写 xdc 程序, 后面发现我手动分配后的 xdc 文件比参考的“多了几行代码”, 正因为者多出的几行代码才能成功烧板。
- 在 CPU 烧板到 basys3 后, 首先需要将 SW_0 置位 1 并手动启动一次脉冲使 PC 清零, 即进行一次 Reset, 否则得不到期望的结果 (部分值没有初始化造成了影响)